



40



27



[原创] 某 PUBG 内核辅助逆向分析  优

一只鸭子



极客  

2026-6-30 13:52

 举报

 40803

某 PUBG 内核辅助逆向分析

最近一个朋友发了一个 PUBG 的内核挂过来，名字就不说了，避免麻烦。久仰外挂隐藏之大名，正好拿来看看现在的内核辅助是怎么写的，用了什么技术、怎么隐藏、怎么和反作弊系统对抗。

样本是一个 `.sh` 自解压脚本，要求 root 权限运行。下面是完整的分析过程。

第一层：Shell 自解压包装

拿到的文件是 `PUBG公益内核.sh`，一个 shell 脚本。打开看头部：

```
#!/system/bin/sh
skip=48
tmpdir=/data/adb/XXXXXX
...
tail +$skip "$0" | gzip -cd > "$tmpdir/ditpro_main"
chmod 755 "$tmpdir/ditpro_main"
"$tmpdir/ditpro_main" &
sleep 5
rm -rf "$0"
```

经典的自解压结构：`tail +48` 跳过前 48 行 shell 头，剩下的是 gzip 压缩的 payload，解压出来就是主程序 `ditpro_main`。

有意思的是最后那个 `sleep 5 && rm -rf "$0"`——执行完 5 秒后把自己删了。不留尸体，取证的时候磁盘上已经没有原始文件了。

写个 Python 脚本把 payload 提取出来：

```
gz_offset = raw.find(b'\x1f\x8b') # gzip magic
gz_data = raw[gz_offset:]
main_bin = gzip.decompress(gz_data)
# ditpro_main.bin: ~40MB, AArch64 PIE ELF
```

解压出来是个 40MB 的 ELF，AArch64 架构，动态链接的 PIE 可执行文件。NDK r27d 编译，target API 23。大得离谱——后面会知道为什么这么大。

第二层：主程序结构概览

把 `ditpro_main.bin` 丢进 IDA，总共 8484 个函数，7937 个没名字（stripped）。段布局值得注意：

段	范围	大小	说明
<code>.rodata</code>	0x5c040 - 0x215278c	34.5 MB	只读数据
<code>.text</code>	0x21b69f0 - 0x26e0018	5.2 MB	代码
<code>.bss</code>	0x26f7240 - 0x272eaa0	225 KB	未初始化数据

`.rodata` 占了 34.5MB，比代码段还大好几倍。这就是那 40MB 体积的来源——所有内核模块的数据都塞在这里面。



40



27



第三层：找到内核模块

逆向外挂最核心的问题：内核驱动在哪？

搜 `.ko` 文件？找不到

最开始尝试在 `binary` 里直接搜 ELF magic `\x7fELF`，搜完发现除了主程序自己的 ELF 头之外，没有第二个 `\x7fELF` 签名。所以 `.ko` 不是以原始二进制形式嵌在里面的。

追字符串线索

换个思路，搜和内核模块相关的字符串。搜 `insmod` 找到了 `sub_21BA524`：

```
// sub_21BA524 - insmod 包装函数
void insmod_and_delete(std::string& ko_path) {
    std::string cmd = "insmod " + ko_path + " > /dev/null 2>&1";
    system(cmd.c_str());
    remove(ko_path.c_str()); // 加载完立刻删文件
}
```

加载完就删，标准的反取证操作。那 `ko_path` 从哪来？往上追调用链。

发现 hex 编码方案

追到 `sub_21BA204`，这个函数负责把数据「解码」成 `.ko` 文件：

```
// sub_21BA204 - hex 解码器
void hex_decode_to_file(std::string& hex_data, std::string& output_path) {
    std::ofstream ofs(output_path);
    for (int i = 0; i < hex_data.length(); i += 2) {
        char hex_pair[3] = { hex_data[i], hex_data[i+1], 0 };
        unsigned char byte = strtoul(hex_pair, NULL, 16);
        ofs.write(&byte, 1);
    }
}
```

每次读 2 个字符，`strtoul` 按十六进制解析，写一个字节。所以 `.ko` 是以 **ASCII hex 字符串** 的形式存储的——不是加密，就是简单的 hex 编码。

比如 ELF 头 `\x7fELF\x02\x01\x01` 在 `.rodata` 里长这样：`7f454c46020101...`

在 IDA 里搜这个 ASCII 字符串，一搜搜出来 17 个。所有的 `.ko` 都找到了。

内核版本路由

再追上一层，到 `sub_21C4898`。这个函数是 `.ko` 的分发入口：

```
void load_kernel_module() {
    FILE* fp = popen("uname -r", "r");
    fgets(kernel_version, 256, fp);
    pclose(fp);

    if (kernel_version starts with "5.10")
        hex_data = &ko_data_510;
    else if (kernel_version starts with "6.1")
        hex_data = &ko_data_61;
    else if (kernel_version starts with "6.6")
        hex_data = &ko_data_66;
    else if (kernel_version == "4.14.117")
        hex_data = &ko_data_414117;
    // ... 17 个版本分支

    hex_decode_to_file(hex_data, random_path);
    system("insmod " + random_path);
    remove(random_path);
}
```



首页



社区



课程



招聘



发现



40



27



先 `uname -r` 拿内核版本，然后 switch/case 选对应版本编译的 .ko。内嵌了 17 个版本，覆盖 4.14.117 到 6.6.87，基本涵盖了主流 Android 设备的内核。

全局变量到内核版本的对应关系：

全局变量	内核版本
xmmword_26F7E00	4.14.117
xmmword_26F7E20	4.14.141
xmmword_26F7E40	4.14.180
xmmword_26F7E60	4.14.186
xmmword_26F7E80	4.19.81
xmmword_26F7EA0	4.19.113
xmmword_26F7F00	4.19.191
xmmword_26F7FA0	5.10
xmmword_26F7FC0	5.15
xmmword_26F7FE0	6.1
xmmword_26F8000	6.6

这些全局变量在 `.init_array` 的 `sub_21CE444` 里被初始化，就是简单的 `memcpy` 把 `.rodata` 里的 hex 字符串拷贝到 `std::string` 对象里。

提取.ko

知道了编码方式，提取就很简单了：

```
def find_hex_ko_blobs(data: bytes):
    pattern = b"7f454c46020101" # ELF magic 的 hex 编码
    results = []
    start = 0
    while True:
        idx = data.find(pattern, start)
        if idx < 0:
            break
        # 读到 null 终止符或非 hex 字符
        end = idx
        while end < len(data) and chr(data[end]) in "0123456789abcdef":
            end += 1
        hex_str = data[idx:end]
        results.append((idx, hex_str))
        start = end + 1
    return results

# 对每个 blob: bytes.fromhex(hex_str) -> .ko 文件
```

提取结果：

```
ko_00_4.19.157_hack.ko    12,040 bytes
ko_01_4.14.180_hack.ko    12,456 bytes
ko_02_6.6.87_hack.ko      28,849 bytes
ko_03_5.4.86_hack.ko      26,248 bytes
ko_05_6.1.129_hack.ko     44,320 bytes
ko_09_4.14.117_hack.ko    12,000 bytes
ko_14_5.15.178_hack.ko    62,056 bytes ← 最大，功能最全
ko_15_5.10.234_hack.ko    30,560 bytes
...共 17 个
```

所有模块名都叫 `hack`，license 声称 GPL，作者 `大大怪`。



首页



社区



课程



招聘



发现



40



27



第四层：内核驱动分析

拿 5.15.178 版本（最大的那个，62KB）丢进 IDA。这次有符号，分析起来舒服多了。

设备注册与自隐藏

`init_module` 做了三件事：

```
__int64 init_module() {
    misc_register(&misc);           // 1. 注册 /dev/niuto01
    list_del_init(&__this_module);  // 2. 从内核模块链表中删除自己
    kobject_del(&module_kobject);   // 3. 从 /sys/module/ 中删除
    return 0;
}
```

`misc` 结构体里写的很清楚：minor 号 255（`MISC_DYNAMIC_MINOR`，动态分配），设备名 `niuto01`，`file_operations` 指向 `dispatch_functions`。

关键是后面两步：`list_del_init` 把自己从内核的模块链表里摘掉，`kobject_del` 把 `/sys/module/hack/` 目录干掉。这样 `lsmod` 看不到、`/sys` 下找不到，但设备节点 `/dev/niuto01` 已经注册好了，`ioctl` 通道照常工作。

配合用户态的 `insmod` 后立即 `remove()` 删文件，三重消失：磁盘上没文件、模块列表里没记录、`sysfs` 里没入口。

ioctl 命令表

`dispatch_ioctl` 是核心，4 个命令：

```
switch (cmd) {
    case 26209: // 读进程内存
        copy_from_user(&req, arg, 0x20);
        ReadProcPhyMem(req.pid, req.addr, req.buf, req.size);
        break;
    case 26210: // 写进程内存
        copy_from_user(&req, arg, 0x20);
        WriteProcPhyMem(req.pid, req.addr, req.buf, req.size);
        break;
    case 26211: // 获取模块基址
        copy_from_user(&req, arg, 0x18);
        copy_from_user(name_buf, req.name_ptr, 0xFF);
        req.result = get_module_base(req.pid, name_buf);
        copy_to_user(arg, &req);
        break;
    case 26212: // 握手校验
        copy_from_user(&req, arg, 0x18);
        req.result = 10086; // magic number
        copy_to_user(arg, &req);
        break;
}
```

26212 是握手命令——用户态打开 `/dev/niuto01` 后先发这个，检查返回值是不是 `10086`，确认驱动已经活着。

物理内存读写——绕过一切

这是整个外挂的技术核心。它不走 `process_vm_readv` 这种正经 API（会被反作弊监控），而是自己手动遍历页表，直接操作物理内存。

第一步：手动页表遍历

`translate_linear_address` 实现了完整的 AArch64 四级页表遍历：



首页



社区



课程



招聘



发现



40



27



```
// AArch64 四级页表: PGD → PUD → PMD → PTE
__int64 translate_linear_address(__int64 mm, unsigned __int64 vaddr) {
    // Level 1: PGD
    pgd = *(mm->pgd + ((vaddr >> 30) & 0x1FF) * 8);
    if (!pgd) return 0;

    // Level 2: PMD (物理地址 → 内核虚拟地址)
    pmd = *((phys_to_virt(pgd)) + ((vaddr >> 21) & 0x1FF) * 8);
    if (!pmd) return 0;

    // 检查是否是 2MB 大页 (Section mapping)
    if (!(pmd & 2)) {
        if (pmd & 1) // bit[0]=1, bit[1]=0 → 大页
            return (pmd & 0xFFFFFFFF000) + (vaddr & 0x1FFFF);
        return 0;
    }

    // Level 3: PTE (4KB 普通页)
    pte = *((phys_to_virt(pmd)) + ((vaddr >> 12) & 0x1FF) * 8);
    if (!(pte & 1)) return 0;
    return (pte & 0xFFFFFFFF000) | (vaddr & 0xFFF);
}
```

其中物理地址到内核虚拟地址的转换用的是： $(phys \& 0x7FFFFFF000) - memstart_addr) | 0xFFFFFFFF8000000000$ ，这是 ARM64 Linux 的线性映射公式。

第二步：物理内存读取

拿到物理地址之后，`read_physical_address` 通过 `ioremap_cache` 把物理地址映射到内核虚拟空间：

```
bool read_physical_address(uint64_t phys_addr, void* user_buf, size_t size) {
    // 通过 mem_section 验证物理页面有效
    if (!pfn_valid(phys_addr >> PAGE_SHIFT))
        return false;

    void* mapped = ioremap_cache(phys_addr, size);
    if (!mapped) return false;

    copy_to_user(user_buf, mapped, size);
    iounmap(mapped);
    return true;
}
```

先检查 `mem_section`（SPARSEMEM 模型下的物理页面有效性校验），然后 `ioremap_cache` 映射、`copy_to_user` 拷贝到用户态、`iounmap` 解除映射。一气呵成。

第三步：跨页读写

`ReadProcPhyMem` 处理了跨页的情况。因为页表是以 4KB 为单位映射的，如果要读的数据跨越了页面边界，就得分多次：

```
while (remaining > 0) {
    page_remaining = 4096 - (vaddr & 0xFFF);
    chunk = min(remaining, page_remaining);

    phys = translate_linear_address(mm, vaddr);
    kernel_va = phys_to_virt(phys);
    copy_to_user(user_buf, kernel_va, chunk);

    vaddr += chunk;
    user_buf += chunk;
    remaining -= chunk;
}
```

注意这个版本更暴力——它直接用 `(phys - memstart_addr) | 0xFFFFFFFF8000000000` 算出内核线性映射地址，连 `ioremap` 都省了。直接当内核指针用，`copy_to_user` 拷走。

模块基址查找

`get_module_base` 用来在目标进程里找某个 `.so` 的加载地址（比如游戏引擎 `libUE4.so`）：

```
__int64 get_module_base(pid_t pid, const char* module_name) {
    task = get_pid_task(find_get_pid(pid));
    mm = get_task_mm(task);

    vma = mm->mmap; // VMA 链表头
    while (vma) {
        if (vma->vm_file) {
            path = file_path(vma->vm_file, buf, 255);
            filename = strrchr(path, '/');
            filename = filename ? filename + 1 : path;
            if (strcmp(filename, module_name) == 0)
                return vma->vm_start; // 找到了，返回基址
        }
        vma = vma->vm_next;
    }
    return 0;
}
```

遍历目标进程的 VMA（Virtual Memory Area）链表，对每个 VMA 用 `file_path` 拿到映射文件路径，`strrchr` 取文件名，和目标模块名比较。匹配上了就返回 `vm_start`。

第五层：用户态对抗技术

内核驱动只是基础设施，用户态的 `ditpro_main` 才是外挂本体。它还用了一堆花活来隐藏自己。

SurfaceFlinger 覆盖层隐藏

外挂要画 ESP（透视框）和准心，但不能让截图、录屏看到。`sub_21BEFEC` 是整个渲染隐藏的初始化函数——3900 字节，144 个基本块，做了一件事：根据 Android 版本动态解析 SurfaceFlinger 的 C++ API。

第一步：获取 Android 版本

```
char version_str[128] = {0};
__system_property_get("ro.build.version.release", version_str);
int android_ver = strtoul(version_str, NULL, 10);

if (android_ver <= 4) {
    __android_log_print(ANDROID_LOG_FATAL, "AImGui",
        "[-] Unsupported system version: %zu", android_ver);
    return;
}
```

日志 tag 是 `AImGui`——Android ImGui 的缩写，整个渲染系统基于 Dear ImGui。版本低于 5 直接退出。

第二步：dlopen 系统库

```
// a2 是一个函数指针表: a2[0]=dlopen, a2[1]=dlsym, a2[2]=dlclose
void* libgui = dlopen("/system/lib64/libgui.so", RTLD_NOW);
void* libutils = dlopen("/system/lib64/libutils.so", RTLD_NOW);
```

没有直接调用 `dlopen`，而是通过传入的函数指针表间接调用。这样 GOT 表里就不会出现 `dlopen` 的记录，增加一点静态分析的难度。

第三步：解析 35+ 个 C++ mangled 符号

这是这个函数的主体。它往一个 288 字节的结构体里填充函数指针，每个指针对应一个 `SurfaceComposerClient` 的方法。结构体布局：



40



27



```
struct SurfaceAPI {
    int64_t    android_version;           // +0
    void*      incStrong;                  // +8    RefBase::incStrong
    void*      decStrong;                  // +16   RefBase::decStrong
    void*      String8_ctor;               // +24   String8::String8(const char*)
    void*      String8_dtor;               // +32   String8::~~String8()
    void*      LayerMetadata_ctor;         // +40
    void*      LayerMetadata_setInt32;     // +48
    void*      SCC_ctor;                   // +56   SurfaceComposerClient()
    void*      createSurface;               // +72   ← 7 个变体，按版本选
    void*      createSurface_and8;         // +80   Android 8 专用
    void*      createSurface_and9;         // +88   Android 9 专用
    void*      mirrorSurface;              // +96   Android 11+
    void*      getInternalDisplayToken;    // +104
    void*      getBuiltInDisplay;          // +112  Android 5-9
    void*      getDisplayState;            // +120  Android 11+
    void*      getDisplayInfo;             // +128  Android 5-11
    void*      getPhysicalDisplayIds;      // +136  Android 10+
    void*      getPhysicalDisplayToken;    // +144  Android 12+
    void*      openGlobalTransaction;      // +152  Android 5-8 (legacy)
    void*      closeGlobalTransaction;     // +160  Android 5-8 (legacy)
    void*      Transaction_ctor;           // +168  Android 12+
    void*      setLayer;                   // +176
    void*      setTrustedOverlay;          // +184  ← 关键：防截图
    void*      setLayerStack;              // +192  Android 13+
    void*      show;                       // +200
    void*      hide;                       // +208
    void*      reparent;                   // +216  Android 12+
    void*      setMatrix;                  // +224
    void*      setPosition;                // +232
    void*      apply;                      // +240
    void*      validate;                   // +248
    void*      getSurface;                  // +256
    void*      SC_disconnect;               // +264  SurfaceControl::disconnect
    void*      SC_setLayer;                 // +272  legacy setLayer
    void*      Surface_disconnect;          // +280  Android 5-6
};
```

最有意思的是 `createSurface` 的版本适配。Android 每个大版本都改过这个函数的签名，所以外挂针对每个版本 `dlsym` 不同的 `mangled name`：

Android 版本	createSurface 签名	变化
5-7	(String8&, uint, uint, int, uint)	基础 5 参数
8	(..., SurfaceControl*, uint, uint)	加父容器 + flags
9	(..., SurfaceControl*, int, int)	参数类型变了
10	(..., SurfaceControl*, LayerMetadata)	加 metadata
11	(..., SurfaceControl*, LayerMetadata, uint*)	加输出参数
12-13	(..., IBinder&, LayerMetadata, uint*)	父容器改为 IBinder
14+	(..., int, int, IBinder&, gui::LayerMetadata, uint*)	gui 命名空间

7 个不同的 C++ 函数签名，7 个不同的 `mangled symbol name`。只要有一个对不上，`dlsym` 返回 `NULL`，`Surface` 就创建不了。这就是为什么函数要这么大——大部分代码都在做版本分发。

事务 API 的版本分裂

Android 9 之前用的是全局事务模型：

```
// Android 5-8: 全局事务
openGlobalTransaction();
surfaceControl->setLayer(0x7FFFFFFF);
closeGlobalTransaction(false);
```



40



27



Android 9+ 改成了 Transaction 对象：

```
// Android 9+: Transaction 对象
Transaction txn;
txn.setLayer(surfaceControl, 0x7FFFFFFF);
txn.setTrustedOverlay(surfaceControl, true); // ← Android 12+
txn.show(surfaceControl);
txn.apply(true); // 或 apply(true, true) on Android 13+
```

连 `Transaction::apply` 的参数都从 Android 13 开始多了一个 bool。外挂专门处理了这个差异：Android 9-12 用 `apply(bool)`，13+ 用 `apply(bool, bool)`。

核心隐藏机制：setTrustedOverlay

```
if (android_ver >= 12) {
    setTrustedOverlay = dlsym(libgui,
        "_ZN7android21SurfaceComposerClient11Transaction"
        "17setTrustedOverlayERKNS_2spINS_14SurfaceControlEEEEb");
}
```

`setTrustedOverlay(surfaceControl, true)` 是 Android 12 引入的 API，本意是让系统 UI（状态栏、导航栏）标记自己为「受信任的覆盖层」。受信任的覆盖层有一个特性：**不会被 SurfaceFlinger 的截图/录屏管线捕获**。

这是系统级别的隐藏——不是 app 层的 `FLAG_SECURE`，而是 SurfaceFlinger 合成器在做帧合成的时候就把它跳过了。所以：

- 反作弊用 `MediaProjection` 截图？看不到
- 反作弊用 `screencap` 命令？看不到
- 反作弊用 SurfaceFlinger 的 `capture` 接口？看不到
- 只有直接拿 framebuffer 或者 GPU 合成后的数据才能看到

配合 `mirrorSurface` (Android 11+)，外挂还能把游戏画面镜像到自己的 Surface 上做分析，而不影响游戏本身的渲染。

补充：ZoomSurface

在字符串里还搜到了 `ZoomSurface` 相关的日志：

```
[*] ZoomSurface called with dsdx: %f, dtdx: %f, dtdy: %f, dsdy: %f
[*] ZoomSurface called with scaleX: %f, scaleY: %f
```

这是通过 `Transaction::setMatrix(surfaceControl, dsdx, dtdx, dtdy, dsdy)` 实现的——对 Surface 做仿射变换（缩放/旋转）。可能用于放大镜功能或者适配不同分辨率。

输入设备伪装

sub_223120C 处理输入注入——外挂需要自动开枪/压枪，但得让触摸事件看起来像真人操作：

```
void setup_input() {
    // 1. 隐藏真实输入设备信息
    system("chmod 000 -R /proc/bus/input/*");

    // 2. 创建虚拟输入设备
    int fd = open("/dev/uinput", O_WRONLY | O_NONBLOCK);
    // 配置为触摸屏设备...

    // 3. 开线程注入事件，加随机延时
    pthread_create(&tid, NULL, input_thread, NULL);
}
```

先把 `/proc/bus/input/` 的权限全干掉，这样反作弊读不到输入设备列表。然后通过 `/dev/uinput` 创建虚拟触摸设备注入事件。注入时加了随机延时，模拟人类操作的不规则性。

ImGui + OpenGL ES 渲染

外挂 UI 用的是 Dear ImGui 1.90.1 + OpenGL ES 3.x + EGL Surface。在 `.rodata` 里能搜到 ImGui 的版本字符串和大量 UI 相关常量。渲染管线：

[社区](#)

[课程](#)

[招聘](#)

[发现](#)



Embree 光线追踪

这个比较少见——binary 里包含了 Intel Embree 光线追踪库的代码。BVH（Bounding Volume Hierarchy）加速结构相关的函数有一大堆。

用途推测是 ESP/透视功能：要判断一个敌人是否「可见」（中间有没有墙），用光线追踪对场景做 ray cast 是最精确的方法。

完整攻击链

把所有层串起来，完整的工作流程：

```
Shell 脚本 (root 执行)
├─ tail + gzip 解压出 ditpro_main
├─ 启动 ditpro_main, 5 秒后删除 .sh 自身
└─
ditpro_main (用户态)
├─
├─ uname -r 获取内核版本
├─ 从 .rodata 选择对应版本的 hex-encoded .ko
├─ hex decode → 写入随机文件名的临时文件
├─ insmod 加载 → 立即 remove() 删除文件
├─
├─   └─ .ko 内核驱动
├─       └─ misc_register("/dev/niuto01")
├─       └─ list_del + kobject_del 从模块列表/sysfs 隐身
├─       └─ 等待 ioctl 指令
├─
└─ open("/dev/niuto01")
```



```
├─ 循环：
├─   └─ ioctl(26209) 读游戏内存（物理内存直接访问）
├─   └─ 计算敌人位置 / 物资位置 / 准心偏移
├─   └─ ioctl(26210) 写游戏内存（修改数值）
├─       └─ ImGui 渲染 ESP / 准心辅助线
├─
├─ SurfaceFlinger: setTrustedOverlay(true) 防截图
├─ chmod 000 /proc/bus/input/* 隐藏输入设备
├─   └─ /dev/uinput 注入触摸事件（自动压枪）
```

隐藏手段总结

层级	技术	目的
文件层	5 秒自删 .sh + insmod 后删 .ko	磁盘无痕
内核层	list_del + kobject_del	lsmod/sysfs 不可见
内存层	物理地址直读，不走 ptrace/process_vm_readv	绕过 API 级监控
渲染层	setTrustedOverlay(true)	截图/录屏不可见
输入层	chmod 000 + uinput 虚拟设备	隐藏注入行为
命名层	随机 40 字符文件名 + stdout 重定向 /dev/null	进程/文件名无特征

后记

从技术角度看，这个外挂的架构还是挺完整的。内核驱动本身很小（最大的也才 62KB），功能单一——就是提供物理内存读写通道。所有复杂逻辑都在用户态，驱动只是个打洞工具。



40



27



最核心的对抗点在于物理内存访问：手动遍历页表 + `ioremap_cache` 这条路完全绕过了内核的访问控制机制，反作弊系统如果只监控 `ptrace`、`/proc/pid/mem`、`process_vm_readv` 这些常规接口，是完全看不到这种读写的。

要检测这类外挂，反作弊至少需要：

- 1. 监控 `misc_register` / `cdev_add` 等设备注册调用
- 2. 定期扫描 `/dev` 下的未知设备节点
- 3. 检测 `ioremap` 调用是否指向了用户态进程的物理页面
- 4. 内核完整性校验——检查模块链表是否被篡改

写一个游戏外挂是难，但是我们可以根据已有的产出进行分析，学习其中对抗和反对抗思路，我也真的很敬佩这些写外挂的大牛，充分发挥逆向精神，让不可能变得可能

附件我感觉不能传了，各位自己可以去搜搜，然后分析一下对抗，现在大部分的内核辅助为了适用性大多数都使用sh来封装一些脚本和二进制

[传递专业知识、拓宽行业人脉——看雪讲师团队等你加入！！](#)



收藏 · 40



点赞 · 27



打赏



分享

赞赏记录

参与人	雪币	留言	时间
mb_wxmrqhty	+1	谢谢你的细致分析，受益匪浅！	2026-8-30 21:28
mb_gzvktfbt		非常支持你的观点！	2026-8-29 01:05
wx_null_277		感谢你的贡献，论坛因你而更加精彩！	2026-8-28 16:42
Andy2023		非常支持你的观点！	2026-8-22 06:17
sk97		期待更多优质内容的分享，论坛有你更精彩！	2026-7-28 16:44
iCookiie		为你点赞！	2026-7-27 16:51
mb_kjajfvq		谢谢你的细致分析，受益匪浅！	2026-7-16 18:47
gamehack		谢谢你的细致分析，受益匪浅！	2026-7-16 10:15
wx_晨梦		你的帖子非常有用，感谢分享！	2026-7-16 06:01
我的小拇指啊		感谢你分享这么好的资源！	2026-7-11 02:23
Ally Switch		谢谢你的细致分析，受益匪浅！	2026-7-10 07:51
mb_fwvkftbc		你的帖子非常有用，感谢分享！	2026-7-8 23:24
没有眼的鱼		这个讨论对我很有帮助，谢谢！	2026-7-8 15:55
漫雾.		你的帖子非常有用，感谢分享！	2026-7-6 14:20
yuanyouran	+1	你的分享对大家帮助很大，非常感谢！	2026-7-6 13:11

孤独的街
首页

感谢你的贡献，论坛因你而更加精彩！
社区

课程

招聘

2026-7-3 16:08
发现



40



27



wx_funcrever

期待更多优质内容的分享，论坛有你更精彩！

2026-7-3 14:47

0xE4s0n

为你点赞！

2026-7-2 11:54

mb_lpcoesnt

+1

感谢你分享这么好的资源！

2026-7-1 19:43

jijo

感谢你分享这么好的资源！

2026-7-1 18:48

[查看更多](#)

最新回复 (20)



[温泉划水鱼](#)

0

[2 楼](#)

牛批

临时

2026-6-30 16:08



0



[小黄鸭爱学习](#)

10

[3 楼](#)

懂了 写挂去了

极客

2026-6-30 17:38



0



[iBa0](#)

10

[4 楼](#)

666

大侠

2026-7-1 20:48



0



[git_14248bibinocode](#)

0

[5 楼](#)

666

临时

2026-7-2 09:38



0



[zylyy](#)

8

[6 楼](#)

666

极客

2026-7-2 22:34



0



[ldjlzlw](#)

11

[7 楼](#)

没有附件我好象看不懂!

极客

2026-7-3 08:51



0



[BitWarden](#)

7

2

[8 楼](#)

还是依旧 内核老套路

大牛

2026-7-3 11:41



0



[JoshuaC](#)

0

[9 楼](#)

附件给个链接谢谢啦~

临时

2026-7-3 15:49



0



首页

社区

课程

招聘

发现

最新回复 (20)

- 

极客

日暮归途 

反作弊第三项和第四项做不到吧。。。。。

2026-7-5 23:31

👍 0

⋮
- 

大牛

BitWarden  

ace的环境监测没做，设备指纹也没做，这种东西能上市吗，不是秒封吗？

2026-7-6 10:39

👍 0

⋮
- 

极客

一只鸭子 

 BitWarden

 ace的环境监测没做，设备指纹也没做，这种东西能上市吗，不是秒封吗？

做设备指纹干啥？都是自己手机用，环境检测这个搞驱动了，说什么无痕了

2026-7-6 12:45

👍 0

⋮



极客

yuanyouran 

学习了

2026-7-6 13:11

👍 0

⋮



大侠

fengyunabc  

感谢分享。

2026-7-8 19:18

👍 0

⋮



大侠

kofly  

mark

2026-7-13 11:57

👍 0

⋮



极客

博士9999 

这个作者一看就是经验老道

2026-7-16 08:13

👍 0

⋮



极客

ATrueMan 

study

2026-7-16 17:19

👍 0

⋮



极客

ATrueMan 

study

2026-7-16 17:20

👍 0

⋮



极客

liowmark 

学习学习

2026-7-17 14:12

👍 0

⋮
-  首页
-  社区
-  课程
-  招聘
-  发现

👍 0

⋮



40



27



最新回复 (20)



[wx_null_277](#)

[20 楼](#)

临时

666666

2026-8-28 16:42

0



[mb_wxmrghty](#)

[21 楼](#)

临时

望尘莫及的逆向还原技术，膜拜大佬

2026-8-30 21:27

0



Baiusc

温馨提示：点击 [引用](#) 按钮再回帖，可回复并 [短消息](#) 提示对方

内容

回帖

表情

高级回复 [MarkDown](#) / [富文本](#)

返回